

Natural language statement

Domain

GroundTruth

Theorem 2.3.1. Let Ω be a regular bounded domain. Assume that for any x in Ω there exists a function $h(x, \cdot)$ in $C^2(\Omega)$ such that $\Delta_y h(x, y) = 0$ in Ω , $h(x, y) = K(x, y)$ for y in $\partial\Omega$. Define the Green's function $G(x, y) = K(x, y) - h(x, y)$ so that, in particular, $\Delta_y G(x, y) = 0$ in Ω , $G(x, y) = 0$ for y in $\partial\Omega$. Then for any $C^2(\Omega)$ -solution u of the Dirichlet problem (2.3.1) and x in Ω we have $u(x) = \int_{\Omega} G(x, y) \Delta_y u(y) dy + \int_{\partial\Omega} u(y) \frac{\partial G(x, y)}{\partial n_y} dy$, where $H(x, y) := -\Delta_y G(x, y)$ in Ω , y in $\partial\Omega$, is the so-called Poisson kernel.

PDE

```

noncomputable def directionalDerivativeList {d : N} :
  List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R
  (Fin d) -> R
| [], u => u
| i :: indices, u => fun x =>
  fderiv R (directionalDerivativeList indices u) x
  (EuclideanSpace.single i 1)

noncomputable def multiIndexDirections {d : N} (alpha : (Fin d -> N)) :
  List (Fin d) :=
  Finset.univ.toList.flatMap fun i => List.replicate (alpha i) i

noncomputable def multiDerivative {d : N} (alpha : (Fin d -> N))
  (u : EuclideanSpace R (Fin d) -> R) : EuclideanSpace R (Fin d) -> R :=
  directionalDerivativeList (multiIndexDirections alpha) u

noncomputable def laplacian {d : N} (u : EuclideanSpace R (Fin d) -> R)
  (x : EuclideanSpace R (Fin d)) : R :=
  sum i, directionalDerivativeList [i, i] u x

def RegularBoundedDomain {d : N} (Omega : Set (EuclideanSpace R (Fin d))) :
  Prop :=
  IsOpen Omega /\ Bornology.IsBounded Omega /\ Omega.Nonempty /\
  forall z in frontier Omega, exists barrier : EuclideanSpace R (Fin d)
  -> R,
  ContinuousOn barrier (closure Omega) /\ ContDiffOn R 2 barrier Omega
  /\
  barrier z = 0 /\ (forall x in closure Omega, x != z -> 0 < barrier x)
  /\
  forall x in Omega, laplacian barrier x <= 0

def IsOutwardUnitNormal {d : N} (Omega : Set (EuclideanSpace R (Fin d)))
  (normal : EuclideanSpace R (Fin d) -> EuclideanSpace R (Fin d)) : Prop
  :=
  forall y in frontier Omega, ||normal y|| = 1 /\
  (forall : R -> EuclideanSpace R (Fin d), forall velocity,
    0 = y -> (forall t in nhds 0, t in frontier Omega) -> HasDerivAt
    velocity 0 ->
    inner R (normal y) velocity = 0) /\
  exists eps : R, 0 < eps /\ y - eps * normal y in Omega /\ y + eps *
  normal y notin closure Omega

def LaplaceDirichletSolution {d : N} (Omega : Set (EuclideanSpace R (Fin
d)))
  (f g u : EuclideanSpace R (Fin d) -> R) : Prop :=
  ContDiffOn R 2 u Omega /\ ContinuousOn u (closure Omega) /\
  (forall x in Omega, laplacian u x = f x) /\ forall x in frontier Omega,
  u x = g x

theorem krylov_2_3_1_green_poisson_representation
  {d : N} {Omega : Set (EuclideanSpace R (Fin d))}
  {K h G H : EuclideanSpace R (Fin d) -> EuclideanSpace R (Fin d) -> R}
  {f g u : EuclideanSpace R (Fin d) -> R}
  (hd : 0 < d) (hOmega : RegularBoundedDomain Omega)
  (hK : IsLaplaceFundamentalSolution K)
  (boundaryMeasure : Measure (EuclideanSpace R (Fin d)))
  (hmeasure : boundaryMeasure = muH[(d : R) - 1].restrict (frontier
Omega))
  (normal : EuclideanSpace R (Fin d) -> EuclideanSpace R (Fin d))
  (hnormal : IsOutwardUnitNormal Omega normal)
  (hharmonic : forall x in Omega, HarmonicIn Omega (h x))
  (hboundary : forall x in Omega, forall y in frontier Omega, h x y = K x
y)
  (hgreen : forall x y, G x y = K x y - h x y)
  (hgreenHarmonic : forall x in Omega, HarmonicIn (Omega \ {x}) (G x))
  (hgreenBoundary : forall x in Omega, forall y in frontier Omega, G x y
= 0)
  (hpoisson : forall x in Omega, forall y in frontier Omega,
  H x y = -fderivWithin R (G x) (closure Omega) y (normal y))
  (hgIntegrable : forall x in Omega, IntegrableOn (fun y => G x y * f y)
Omega)
  (hHintegrable : forall x in Omega, Integrable (fun y => H x y * g y)

```

Natural language statement

Domain

GroundTruth

		<pre> boundaryMeasure) (hu : LaplaceDirichletSolution Omega f g u) : forall x in Omega, u x = integral y in Omega, G x y * f y + integral y, H x y * g y dboundaryMeasure := by sorry </pre>
Theorem 2.5.2. Let Ω be a domain and u in $C^2_{\text{loc}}(\Omega)$ inter $C(\Omega)$ be a harmonic function in Ω. Then u is infinitely differentiable in Ω and for any multi-index α and any x in Ω we have $D^\alpha u(x) \leq N R^{n- \alpha } \sup_{B_R(x)} u$ whenever $B_R(x) \subset \Omega$.	PDE	<pre> noncomputable def directionalDerivativeList {d : N} : List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R [], u => u i :: indices, u => fun x => fderiv R (directionalDerivativeList indices u) x (EuclideanSpace.single i 1) noncomputable def multiIndexDirections {d : N} (alpha : (Fin d -> N)) : List (Fin d) := Finset.univ.toList.flatMap fun i => List.replicate (alpha i) i noncomputable def multiDerivative {d : N} (alpha : (Fin d -> N)) (u : EuclideanSpace R (Fin d) -> R) : EuclideanSpace R (Fin d) -> R := directionalDerivativeList (multiIndexDirections alpha) u noncomputable def laplacian {d : N} (u : EuclideanSpace R (Fin d) -> R) (x : EuclideanSpace R (Fin d)) : R := sum i, directionalDerivativeList [i, i] u x def HarmonicIn {d : N} (Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d) -> R) : Prop := ContDiffOn R 2 u Omega /\ forall x in Omega, laplacian u x = 0 theorem krylov_2_5_2_harmonic_smooth_interior_estimates {d : N} : forall alpha : (Fin d -> N), exists C : R, 0 <= C /\ forall (Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d) -> R), IsOpen Omega -> IsConnected Omega -> Omega.Nonempty -> HarmonicIn Omega u -> ContDiffOn R inf u Omega /\ forall x in Omega, forall R : R, 0 < R -> Metric.closedBall x R subset Omega -> multiDerivative alpha u x <= C * R ^ (-(sum i, alpha i : Z)) * sSup { u y y in Metric.closedBall x R} := by sorry </pre>
Theorem 2.9.2. Let Ω be a domain in $\mathbb{R}^d$ and u be a bounded and continuous function on Ω and $u = 0$ on $\partial\Omega$ if $\partial\Omega \neq \emptyset$ (that is, if $\Omega \neq \mathbb{R}^d$). Moreover, assume that u in $C^2_{\text{loc}}(\Omega)$. Finally, let $a(x)$, $b(x)$ be bounded and $c(x) \leq -\lambda$ where the constant $\lambda > 0$. Then in Ω, $u \leq \lambda b^{-1} \sup_{\Omega} (Lu)^+$ and $u \leq \lambda b^{-1} \sup_{\Omega} Lu$.	PDE	<pre> noncomputable def directionalDerivativeList {d : N} : List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R [], u => u i :: indices, u => fun x => fderiv R (directionalDerivativeList indices u) x (EuclideanSpace.single i 1) noncomputable def multiIndexDirections {d : N} (alpha : (Fin d -> N)) : List (Fin d) := Finset.univ.toList.flatMap fun i => List.replicate (alpha i) i noncomputable def multiDerivative {d : N} (alpha : (Fin d -> N)) (u : EuclideanSpace R (Fin d) -> R) : EuclideanSpace R (Fin d) -> R := directionalDerivativeList (multiIndexDirections alpha) u structure EllipticOperatorData {d : N} (m : N) (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) where terms : Finset ((Fin d -> N)) order_le : forall alpha in terms, sum i, alpha i <= m coefficient : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R formula : forall u x, L u x = sum alpha in terms, coefficient alpha x * multiDerivative alpha u x ellipticityConstant : R ellipticityConstant_pos : 0 < ellipticityConstant principalSymbol : forall (x xi : EuclideanSpace R (Fin d)), ellipticityConstant * xi ^ m <= </pre>

Natural language statement

Domain

GroundTruth

		<pre> sum alpha in terms with sum i, alpha i = m, coefficient alpha x * prod i, (xi i) ^ (alpha i) def SecondOrderEllipticOperator {d : N} (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) (lam : R) : Prop := exists data : EllipticOperatorData 2 L, exists zeroIndex in data.terms, (forall i, zeroIndex i = 0) /\ (forall x, data.coefficient zeroIndex x <= -lam) /\ exists C : R, forall alpha in data.terms, forall x, data.coefficient alpha x <= C noncomputable def functionSupNorm {X : Type*} (Omega : Set X) (u : X -> R) : R>=0inf := x : Omega, ENNReal.ofReal u x theorem krylov_2_9_2_bounded_maximum_principle_resolvent {d : N} {Omega : Set (EuclideanSpace R (Fin d))} {L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R} {lam : R} {u : EuclideanSpace R (Fin d) -> R} (hOmega : IsOpen Omega) (huDiff : ContDiffOn R 2 u Omega) (huContinuous : ContinuousOn u (closure Omega)) (huBounded : Bornology.IsBounded (u '' Omega)) (huBoundary : forall x in frontier Omega, u x = 0) (hlam : 0 < lam) (hL : SecondOrderEllipticOperator L lam) : functionSupNorm Omega (fun x => max (u x) 0) <= (ENNReal.ofReal lam)^-1 * functionSupNorm Omega (fun x => max (- (L u x)) 0) /\ functionSupNorm Omega u <= (ENNReal.ofReal lam)^-1 * functionSupNorm Omega (L u) := by sorry </pre>
Theorem 3.7.2. Let $\lambda \neq 0$, $k \geq 0$ be an integer and $0 < \delta < 1$. Then for any f in $C^{k+\delta}(\mathbb{R}^d)$ there exists a unique solution u in $C^{k+m+\delta}(\mathbb{R}^d)$ of the equation $L_\lambda u(x) = f(x)$, x in $\mathbb{R}^d$.	PDE	<pre> noncomputable def directionalDerivativeList {d : N} : List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R [], u => u i :: indices, u => fun x => fderiv R (directionalDerivativeList indices u) x (EuclideanSpace.single i 1) noncomputable def multiIndexDirections {d : N} (alpha : (Fin d -> N)) : List (Fin d) := Finset.univ.toList.flatMap fun i => List.replicate (alpha i) i noncomputable def multiDerivative {d : N} (alpha : (Fin d -> N)) (u : EuclideanSpace R (Fin d) -> R) : EuclideanSpace R (Fin d) -> R := directionalDerivativeList (multiIndexDirections alpha) u noncomputable def holderGauge {d : N} (k : N) (delta : R) (Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d) -> R) : R>=0inf := (alpha : {alpha : (Fin d -> N) // sum i, alpha i <= k}, x : Omega, ENNReal.ofReal multiDerivative alpha u x) alpha : {alpha : (Fin d -> N) // sum i, alpha i = k}, x : Omega, y : Omega, (_ : (x : EuclideanSpace R (Fin d)) != y), ENNReal.ofReal (multiDerivative alpha u x - multiDerivative alpha u y / (x : EuclideanSpace R (Fin d)) - y ^ delta) def HolderOn {d : N} (r : R) (Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d) -> R) : Prop := exists k : N, exists delta : R, r = k + delta /\ 0 <= delta /\ delta < 1 /\ ContDiffOn R k u Omega /\ holderGauge k delta Omega u < top structure EllipticOperatorData {d : N} (m : N) where terms : Finset ((Fin d -> N)) order_le : forall alpha in terms, sum i, alpha i <= m </pre>

Natural language statement

Domain

GroundTruth

		<pre> coefficient : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R formula : forall u x, L u x = sum alpha in terms, coefficient alpha x * multiDerivative alpha u x ellipticityConstant : R ellipticityConstant_pos : 0 < ellipticityConstant principalSymbol : forall (x xi : EuclideanSpace R (Fin d)), ellipticityConstant * xi ^ m <= sum alpha in terms with sum i, alpha i = m, coefficient alpha x * prod i, (xi i) ^ (alpha i) def ConstantCoefficientEllipticOperator {d : N} (m : N) (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) : Prop := exists data : EllipticOperatorData m L, forall alpha in data.terms, forall x y, data.coefficient alpha x = data.coefficient alpha y def ShiftedEllipticEquation {d : N} (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) (lam : R) (u f : EuclideanSpace R (Fin d) -> R) : Prop := forall x, L u x - lam * u x = f x theorem krylov_3_7_2_constant_coefficient_holder_solvability {d m k : N} {delta lam : R} {L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R} (hm : 0 < m) (hdelta : 0 < delta /\ delta < 1) (hlam : 0 < lam) (hL : ConstantCoefficientEllipticOperator m L) : forall f, HolderOn (k + delta) univ f -> exists! u, HolderOn (k + m + delta) univ u /\ ShiftedEllipticEquation L lam u f := by sorry </pre>
Theorem 4.2.1. Let the assumptions of Theorem 4.1.2 be satisfied, and let $k \geq 0$ be an integer, $K_1 \geq 1$ be a constant. Assume that for any α we have $a^\alpha _{k+\delta} \leq K_1$. Then for any λ, the inclusions $u \in C^{m+\delta}(R^d)$ and $L_\lambda u \in C^{k+\delta}(R^d)$ imply that $u \in C^{k+m+\delta}(R^d)$. Moreover, if we take $\lambda_{\text{bda}0}$ from Theorem 4.1.2 and take a real λ so that $\lambda_{\text{bda}} \geq \lambda_{\text{bda}0}$, then there exists a constant $N > 0$ depending only on $\kappa, k, m, \delta, K_1, d$, such that for any u in $C^{k+m+\delta}(R^d)$ we have $u _{k+m+\delta} + \lambda_{\text{bda}} ^{((k+m+\delta)/m)} u _0 \leq N(L_\lambda u _{k+\delta} + \lambda_{\text{bda}} ^{((k+\delta)/m)} L_\lambda u _0)$.	PDE	<pre> noncomputable def directionalDerivativeList {d : N} : List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R [], u => u i :: indices, u => fun x => fderiv R (directionalDerivativeList indices u) x (EuclideanSpace.single i 1) noncomputable def multiIndexDirections {d : N} (alpha : (Fin d -> N)) : List (Fin d) := Finset.univ.toList.flatMap fun i => List.replicate (alpha i) i noncomputable def multiDerivative {d : N} (alpha : (Fin d -> N)) (u : EuclideanSpace R (Fin d) -> R) : EuclideanSpace R (Fin d) -> R := directionalDerivativeList (multiIndexDirections alpha) u noncomputable def holderGauge {d : N} (k : N) (delta : R) (Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d) -> R) : R => 0inf := (alpha : {alpha : (Fin d -> N) // sum i, alpha i <= k}, x : Omega, ENNReal.ofReal multiDerivative alpha u x) alpha : {alpha : (Fin d -> N) // sum i, alpha i = k}, x : Omega, y : Omega, (_ : (x : EuclideanSpace R (Fin d)) != y), ENNReal.ofReal (multiDerivative alpha u x - multiDerivative alpha u y / (x : EuclideanSpace R (Fin d)) - y ^ delta) def HolderOn {d : N} (r : R) (Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d) -> R) : Prop := exists k : N, exists delta : R, r = k + delta /\ 0 <= delta /\ delta < 1 /\ ContDiffOn R k u Omega /\ holderGauge k delta Omega u < top structure EllipticOperatorData {d : N} (m : N) (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) where terms : Finset ((Fin d -> N)) order_le : forall alpha in terms, sum i, alpha i <= m </pre>

Natural language statement

Domain

GroundTruth

		<pre> coefficient : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R formula : forall u x, L u x = sum alpha in terms, coefficient alpha x * multiDerivative alpha u x ellipticityConstant : R ellipticityConstant_pos : 0 < ellipticityConstant principalSymbol : forall (x xi : EuclideanSpace R (Fin d)), ellipticityConstant * xi ^ m <= sum alpha in terms with sum i, alpha i = m, coefficient alpha x * prod i, (xi i) ^ (alpha i) def VariableCoefficientEllipticOperator {d : N} (m : N) (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) : Prop := Nonempty (EllipticOperatorData m L) def OperatorCoefficientsHolder {d : N} (m : N) (r : R) (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) : Prop := exists data : EllipticOperatorData m L, forall alpha in data.terms, HolderOn r univ (data.coefficient alpha) def OperatorCoefficientGaugeLE {d : N} (m k : N) (delta : R) (K : R>=0inf) (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) : Prop := exists data : EllipticOperatorData m L, forall alpha in data.terms, holderGauge k delta univ (data.coefficient alpha) <= K def ShiftedEllipticEquation {d : N} (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) (lam : R) (u f : EuclideanSpace R (Fin d) -> R) : Prop := forall x, L u x - lam * u x = f x theorem krylov_4_2_1_better_regular_data_better_regular_solution {d m k : N} {delta K1 : R} {L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R} (hm : 0 < m) (hdelta : 0 < delta /\ delta < 1) (hK1 : 1 <= K1) (hL : VariableCoefficientEllipticOperator m L) (hcoeff : OperatorCoefficientsHolder m (k + delta) L) (hcoeffBound : OperatorCoefficientGaugeLE m k delta (ENNReal.ofReal K1) L) : exists lam0 : R, 0 < lam0 /\ exists C : R>=0inf, C < inf /\ forall (lam : R) (u f : EuclideanSpace R (Fin d) -> R), HolderOn (m + delta) univ u -> ShiftedEllipticEquation L lam u f -> HolderOn (k + delta) univ f -> HolderOn (k + m + delta) univ u /\ (lam0 <= lam -> holderGauge (k + m) delta univ u + ENNReal.rpow (ENNReal.ofReal lam) (((k + m : N) : R) + delta) / m) * functionSupNorm univ u <= C * (holderGauge k delta univ f + ENNReal.rpow (ENNReal.ofReal lam) (((k : N) : R) + delta) / m) * functionSupNorm univ f)) := by sorry </pre>
Theorem 4.5.1. Let $L = L(x) = \sum_{ \alpha \leq m} a^\alpha(x) D^\alpha$ be a uniformly elliptic operator, and $k \geq 0$ be an integer. Assume that a^α in $C^{k+\delta}(R^d)$ for any α. Define the constant $\lambda_{\delta,0}$ depending only on the ellipticity constant κ and m, δ, d and maximum of $a^\alpha _{\delta}$ as in Theorem 4.1.2, and take any real λ_{δ} such that $\lambda_{\delta} \geq \lambda_{\delta,0}$. Then for any f in $C^{k+\delta}(R^d)$ there exists a unique solution u in $C^{k+m+\delta}(R^d)$ of the equation $L_{\lambda_{\delta}} u(x) = f(x)$, x in R^d.	PDE	<pre> noncomputable def directionalDerivativeList {d : N} : List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R [], u => u i :: indices, u => fun x => fderiv R (directionalDerivativeList indices u) x (EuclideanSpace.single i 1) noncomputable def multiIndexDirections {d : N} (alpha : (Fin d -> N)) : List (Fin d) := Finset.univ.toList.flatMap fun i => List.replicate (alpha i) i noncomputable def multiDerivative {d : N} (alpha : (Fin d -> N)) (u : EuclideanSpace R (Fin d) -> R) : EuclideanSpace R (Fin d) -> R := </pre>

Natural language statement

Domain

GroundTruth

		<pre> directionalDerivativeList (multiIndexDirections alpha) u noncomputable def holderGauge {d : N} (k : N) (delta : R) (Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d) -> R) : R := 0 inf := (alpha : {alpha : (Fin d -> N) // sum i, alpha i <= k}, x : Omega, ENNReal.ofReal multiDerivative alpha u x) alpha : {alpha : (Fin d -> N) // sum i, alpha i = k}, x : Omega, y : Omega, (_ : (x : EuclideanSpace R (Fin d)) != y), ENNReal.ofReal (multiDerivative alpha u x - multiDerivative alpha u y / (x : EuclideanSpace R (Fin d)) - y ^ delta) def HolderOn {d : N} (r : R) (Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d) -> R) : Prop := exists k : N, exists delta : R, r = k + delta /\ 0 <= delta /\ delta < 1 /\ ContDiffOn R k u Omega /\ holderGauge k delta Omega u < top structure EllipticOperatorData {d : N} (m : N) (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) where terms : Finset ((Fin d -> N)) order_le : forall alpha in terms, sum i, alpha i <= m coefficient : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R formula : forall u x, L u x = sum alpha in terms, coefficient alpha x * multiDerivative alpha u x ellipticityConstant : R ellipticityConstant_pos : 0 < ellipticityConstant principalSymbol : forall (x xi : EuclideanSpace R (Fin d)), ellipticityConstant * xi ^ m <= sum alpha in terms with sum i, alpha i = m, coefficient alpha x * prod i, (xi i) ^ (alpha i) def VariableCoefficientEllipticOperator {d : N} (m : N) (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) : Prop := Nonempty (EllipticOperatorData m L) def OperatorCoefficientsHolder {d : N} (m : N) (r : R) (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) : Prop := exists data : EllipticOperatorData m L, forall alpha in data.terms, HolderOn r univ (data.coefficient alpha) def ShiftedEllipticEquation {d : N} (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) (lam : R) (u f : EuclideanSpace R (Fin d) -> R) : Prop := forall x, L u x - lam * u x = f x theorem krylov_4_5_1_variable_coefficient_global_solvability {d m k : N} {delta : R} {L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R} (hm : 0 < m) (hdelta : 0 < delta /\ delta < 1) (hL : VariableCoefficientEllipticOperator m L) (hcoeff : OperatorCoefficientsHolder m (k + delta) L) : exists lam0 : R, 0 < lam0 /\ forall lam : R, lam0 <= lam -> forall f, HolderOn (k + delta) univ f -> exists! u, HolderOn (k + m + delta) univ u /\ ShiftedEllipticEquation L lam u f := by sorry </pre>
Theorem 6.5.3. For any f in $C^{k+\delta}(\Omega)$ and g in $C^{k+2+\delta}(\Omega)$ there exists a unique function u in $C^{k+2+\delta}(\Omega)$ satisfying the equation $Lu = f$ in Ω and equal to g on $\partial\Omega$.	PDE	<pre> noncomputable def directionalDerivativeList {d : N} : List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R [], u => u i :: indices, u => fun x => fderiv R (directionalDerivativeList indices u) x (EuclideanSpace.single i 1) </pre>

Natural language statement

Domain

GroundTruth

```

noncomputable def multiIndexDirections {d : N} (alpha : (Fin d -> N)) :
List (Fin d) :=
  Finset.univ.toList.flatMap fun i => List.replicate (alpha i) i

noncomputable def multiDerivative {d : N} (alpha : (Fin d -> N))
(u : EuclideanSpace R (Fin d) -> R) : EuclideanSpace R (Fin d) -> R :=
directionalDerivativeList (multiIndexDirections alpha) u

noncomputable def holderGauge {d : N} (k : N) (delta : R)
(Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d)
-> R) : R:=0inf :=
  ( alpha : {alpha : (Fin d -> N) // sum i, alpha i <= k},
    x : Omega, ENNReal.ofReal |multiDerivative alpha u x|)
  alpha : {alpha : (Fin d -> N) // sum i, alpha i = k},
  x : Omega, y : Omega, ( _ : (x : EuclideanSpace R (Fin d)) != y),
  ENNReal.ofReal
  (|multiDerivative alpha u x - multiDerivative alpha u y| /
    ||(x : EuclideanSpace R (Fin d)) - y|| ^ delta)

def HolderOn {d : N} (r : R) (Omega : Set (EuclideanSpace R (Fin d)))
(u : EuclideanSpace R (Fin d) -> R) : Prop :=
exists k : N, exists delta : R, r = k + delta /\ 0 <= delta /\ delta < 1
/\
  ContDiffOn R k u Omega /\ holderGauge k delta Omega u < top

def SmoothBoundedDomain {d : N} (Omega : Set (EuclideanSpace R (Fin d))) :
Prop :=
  RegularBoundedDomain Omega /\ exists rho : EuclideanSpace R (Fin d) -> R,
  ContDiff R inf rho /\ Omega = {x | rho x < 0} /\
  forall x in frontier Omega, fderiv R rho x != 0

structure EllipticOperatorData {d : N} (m : N)
  (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R)
  where
  terms : Finset ((Fin d -> N))
  order_le : forall alpha in terms, sum i, alpha i <= m
  coefficient : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R
  formula : forall u x, L u x = sum alpha in terms, coefficient alpha x *
    multiDerivative alpha u x
  ellipticityConstant : R
  ellipticityConstant_pos : 0 < ellipticityConstant
  principalSymbol : forall (x xi : EuclideanSpace R (Fin d)),
    ellipticityConstant * ||xi|| ^ m <=
      sum alpha in terms with sum i, alpha i = m,
      coefficient alpha x * prod i, (xi i) ^ (alpha i)

def VariableCoefficientEllipticOperator {d : N} (m : N)
  (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R)
  : Prop :=
  Nonempty (EllipticOperatorData m L)

def EllipticDirichletSolution {d : N} (Omega : Set (EuclideanSpace R (Fin
d)))
  (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R)
  (f g u : EuclideanSpace R (Fin d) -> R) : Prop :=
  ContDiffOn R 2 u Omega /\ ContinuousOn u (closure Omega) /\
  (forall x in Omega, L u x = f x) /\ forall x in frontier Omega, u x = g
  x

theorem krylov_6_5_3_smooth_domain_dirichlet_solvability
{d k : N} {delta : R} {Omega : Set (EuclideanSpace R (Fin d))}
{L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R}
(hdelta : 0 < delta /\ delta < 1) (hOmega : SmoothBoundedDomain Omega)
(hL : SecondOrderEllipticOperator L 0)
(hcoeff : OperatorCoefficientsHolder 2 (k + delta) L) :
forall f g, HolderOn (k + delta) Omega f -> HolderOn (k + 2 + delta)
(closure Omega) g ->
  exists u, HolderOn (k + 2 + delta) Omega u /\
  EllipticDirichletSolution Omega L f g u /\
  forall v, HolderOn (k + 2 + delta) Omega v ->
  EllipticDirichletSolution Omega L f g v ->
  Set.EqOn v u (closure Omega) := by

```

Natural language statement

Domain

GroundTruth

		sorry
Theorem 7.1.2. Let Ω be a domain in $\mathbb{R}^d$ and u in $C^{(m+\delta)}(\Omega)$. Assume that L_λ in $C^{(k+\delta)}(\Omega)$ for some λ. Then u in $C^{(k+m+\delta)}(\Omega)$.	PDE	<pre> noncomputable def directionalDerivativeList {d : N} : List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R [], u => u i :: indices, u => fun x => fderiv R (directionalDerivativeList indices u) x (EuclideanSpace.single i 1) noncomputable def multiIndexDirections {d : N} (alpha : (Fin d -> N)) : List (Fin d) := Finset.univ.toList.flatMap fun i => List.replicate (alpha i) i noncomputable def multiDerivative {d : N} (alpha : (Fin d -> N)) (u : EuclideanSpace R (Fin d) -> R) : EuclideanSpace R (Fin d) -> R := directionalDerivativeList (multiIndexDirections alpha) u noncomputable def holderGauge {d : N} (k : N) (delta : R) (Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d) -> R) : R>=0inf := (alpha : {alpha : (Fin d -> N) // sum i, alpha i <= k}, x : Omega, ENNReal.ofReal [multiDerivative alpha u x]) alpha : {alpha : (Fin d -> N) // sum i, alpha i = k}, x : Omega, y : Omega, (_ : (x : EuclideanSpace R (Fin d)) != y), ENNReal.ofReal ([multiDerivative alpha u x - multiDerivative alpha u y] / (x : EuclideanSpace R (Fin d)) - y ^ delta) def HolderOn {d : N} (r : R) (Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d) -> R) : Prop := exists k : N, exists delta : R, r = k + delta /\ 0 <= delta /\ delta < 1 /\ ContDiffOn R k u Omega /\ holderGauge k delta Omega u < top structure EllipticOperatorData {d : N} (m : N) (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) where terms : Finset ((Fin d -> N)) order_le : forall alpha in terms, sum i, alpha i <= m coefficient : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R formula : forall u x, L u x = sum alpha in terms, coefficient alpha x * multiDerivative alpha u x ellipticityConstant : R ellipticityConstant_pos : 0 < ellipticityConstant principalSymbol : forall (x xi : EuclideanSpace R (Fin d)), ellipticityConstant * xi ^ m <= sum alpha in terms with sum i, alpha i = m, coefficient alpha x * prod i, (xi i) ^ (alpha i) def ShiftedEllipticEquation {d : N} (L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R) (lam : R) (u f : EuclideanSpace R (Fin d) -> R) : Prop := forall x, L u x - lam * u x = f x theorem krylov_7_1_2_interior_holder_regularization {d m k : N} {delta lam : R} {Omega : Set (EuclideanSpace R (Fin d))} {L : (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R (Fin d) -> R} {u f : EuclideanSpace R (Fin d) -> R} (hdelta : 0 < delta /\ delta < 1) (hOmega : IsOpen Omega) (hL : VariableCoefficientEllipticOperator m L) (hcoeff : OperatorCoefficientsHolder m (k + delta) L) (hu : HolderLocallyOn (m + delta) Omega u) (hLu : ShiftedEllipticEquationOn Omega L lam u f) (hf : HolderLocallyOn (k + delta) Omega f) : HolderLocallyOn (k + m + delta) Omega u := by sorry </pre>
Theorem 8.7.3. For any f in $C^{(\delta/2, \delta)}(\mathbb{R}^{(d+1)})$ there exists a unique function u in $C^{(1+\delta/2, 2+\delta)}(\mathbb{R}^{(d+1)})$	PDE	<pre> noncomputable def directionalDerivativeList {d : N} : List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R </pre>

Natural language statement

Domain

GroundTruth

satisfying the equation $\Delta u - u = f$ in R^{d+1} .

```
(Fin d) -> R
| [], u => u
| i :: indices, u => fun x =>
  fderiv R (directionalDerivativeList indices u) x
  (EuclideanSpace.single i 1)

noncomputable def multiIndexDirections {d : N} (alpha : (Fin d -> N)) :
List (Fin d) :=
  Finset.univ.toList.flatMap fun i => List.replicate (alpha i) i

noncomputable def multiDerivative {d : N} (alpha : (Fin d -> N))
(u : EuclideanSpace R (Fin d) -> R) : EuclideanSpace R (Fin d) -> R :=
  directionalDerivativeList (multiIndexDirections alpha) u

noncomputable def laplacian {d : N} (u : EuclideanSpace R (Fin d) -> R)
(x : EuclideanSpace R (Fin d)) : R :=
  sum i, directionalDerivativeList [i, i] u x

def HolderOnReal (r : R) (I : Set R) (u : R -> R) : Prop :=
  exists (k : N) (delta : R) (hdelta : 0 <= delta), r = k + delta /\ delta
  < 1 /\ ContDiffOn R k u I /\
  exists C : NNReal, HolderOnWith C (<delta, hdelta) : NNReal
  (iteratedDeriv k u) I

def ParabolicHolderOn {d : N} (r : R) (Q : Set (R x EuclideanSpace R (Fin
d)))
(u : (R x EuclideanSpace R (Fin d)) -> R) : Prop :=
  (forall t, HolderOn r {x | (t, x) in Q} fun x => u (t, x)) /\
  forall x, HolderOnReal (r / 2) {t | (t, x) in Q} fun t => u (t, x)

def ShiftedHeatEquation {d : N} (u f : (R x EuclideanSpace R (Fin d)) ->
R) : Prop :=
  forall t x, laplacian (fun y => u (t, y)) x - deriv (fun s => u (s, x)) t
  - u (t, x) =
  f (t, x)

theorem krylov_8_7_3_shifted_heat_holder_solvability
{d : N} {delta : R} (hdelta : 0 < delta /\ delta < 1) :
forall f : (R x EuclideanSpace R (Fin d)) -> R, ParabolicHolderOn
delta univ f ->
  exists! u : (R x EuclideanSpace R (Fin d)) -> R,
  ParabolicHolderOn (2 + delta) univ u /\ ShiftedHeatEquation u f :=
  by
  sorry
```

Theorem 10.3.3. For any f in $C^{\lfloor \frac{\delta}{2} \rfloor}(Q)$ and g in $C^{\lfloor 1 + \frac{\delta}{2} \rfloor}(Q)$ there exists a unique function u in $C^{\lfloor 1 + \frac{\delta}{2} \rfloor}(Q)$ satisfying the equation $\Delta u - u = f$ in Q and equal g on dQ .

PDE

```
noncomputable def directionalDerivativeList {d : N} :
List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R
(Fin d) -> R
| [], u => u
| i :: indices, u => fun x =>
  fderiv R (directionalDerivativeList indices u) x
  (EuclideanSpace.single i 1)

noncomputable def multiIndexDirections {d : N} (alpha : (Fin d -> N)) :
List (Fin d) :=
  Finset.univ.toList.flatMap fun i => List.replicate (alpha i) i

noncomputable def multiDerivative {d : N} (alpha : (Fin d -> N))
(u : EuclideanSpace R (Fin d) -> R) : EuclideanSpace R (Fin d) -> R :=
  directionalDerivativeList (multiIndexDirections alpha) u

noncomputable def holderGauge {d : N} (k : N) (delta : R)
(Omega : Set (EuclideanSpace R (Fin d))) (u : EuclideanSpace R (Fin d)
-> R) : R :=
  0inf :=
  ( alpha : {alpha : (Fin d -> N) // sum i, alpha i <= k},
    x : Omega, ENNReal.ofReal |multiDerivative alpha u x|)
  alpha : {alpha : (Fin d -> N) // sum i, alpha i = k},
  x : Omega, y : Omega, ( _ : (x : EuclideanSpace R (Fin d)) != y),
  ENNReal.ofReal
  (|multiDerivative alpha u x - multiDerivative alpha u y| /
  |(x : EuclideanSpace R (Fin d)) - y| ^ delta)
```

Natural language statement

Domain

GroundTruth

```

def HolderOnReal (r : R) (I : Set R) (u : R -> R) : Prop :=
  exists (k : N) (delta : R) (hdelta : 0 <= delta), r = k + delta /\ delta
  < 1 /\ ContDiffOn R k u I /\
  exists C : NNReal, HolderOnWith C (<delta, hdelta> : NNReal)
  (iteratedDeriv k u) I

def ParabolicHolderOn {d : N} (r : R) (Q : Set (R x EuclideanSpace R (Fin d)))
(u : (R x EuclideanSpace R (Fin d)) -> R) : Prop :=
  (forall t, HolderOn r {x | (t, x) in Q} fun x => u (t, x)) /\
  forall x, HolderOnReal (r / 2) {t | (t, x) in Q} fun t => u (t, x)

def ParabolicOperator {d : N}
(L : ((R x EuclideanSpace R (Fin d)) -> R) ->
  (R x EuclideanSpace R (Fin d)) -> R) : Prop :=
  exists (a : (R x EuclideanSpace R (Fin d)) -> Matrix (Fin d) (Fin d) R)
  (b : (R x EuclideanSpace R (Fin d)) -> EuclideanSpace R (Fin d))
  (c : (R x EuclideanSpace R (Fin d)) -> R) (kappa : R),
  0 < kappa /\
  (forall (p : (R x EuclideanSpace R (Fin d))) (xi : EuclideanSpace R
  (Fin d)),
    kappa * ||xi|| ^ 2 <= sum i, sum j, a p i j * xi i * xi j) /\
  forall u p, L u p =
    sum i, sum j, a p i j *
      directionalDerivativeList [i, j] (fun x => u (p.1, x)) p.2 +
    sum i, b p i * fderiv R (fun x => u (p.1, x)) p.2
    (EuclideanSpace.single i 1) +
    c p * u p

def ParabolicOperatorCoefficientsHolder {d : N} (r : R)
(Q : Set (R x EuclideanSpace R (Fin d)))
(L : ((R x EuclideanSpace R (Fin d)) -> R) ->
  (R x EuclideanSpace R (Fin d)) -> R) : Prop :=
  exists (a : (R x EuclideanSpace R (Fin d)) -> Matrix (Fin d) (Fin d) R)
  (b : (R x EuclideanSpace R (Fin d)) -> EuclideanSpace R (Fin d))
  (c : (R x EuclideanSpace R (Fin d)) -> R),
  (forall i j, ParabolicHolderOn r Q fun p => a p i j) /\
  (forall i, ParabolicHolderOn r Q fun p => b p i) /\ ParabolicHolderOn
  r Q c /\
  forall u p, L u p =
    sum i, sum j, a p i j *
      directionalDerivativeList [i, j] (fun x => u (p.1, x)) p.2 +
    sum i, b p i * fderiv R (fun x => u (p.1, x)) p.2
    (EuclideanSpace.single i 1) +
    c p * u p

def parabolicBoundary {d : N} (Q : Set (R x EuclideanSpace R (Fin d))) :
  Set (R x EuclideanSpace R (Fin d)) :=
  {p in frontier Q | forall eps : R, 0 < eps ->
    exists q in Q, p.1 <= q.1 /\ dist q p < eps}

def ParabolicDirichletSolution {d : N} (Q : Set (R x EuclideanSpace R
  (Fin d)))
(L : ((R x EuclideanSpace R (Fin d)) -> R) ->
  (R x EuclideanSpace R (Fin d)) -> R)
(f g u : (R x EuclideanSpace R (Fin d)) -> R) : Prop :=
  ContinuousOn u (Q union parabolicBoundary Q) /\
  (forall p in Q, DifferentiableAt R (fun t => u (t, p.2)) p.1) /\
  (forall p in Q, L u p - deriv (fun t => u (t, p.2)) p.1 = f p) /\
  forall p in parabolicBoundary Q, u p = g p

theorem krylov_10_3_3_parabolic_dirichlet_domain_solvability
{d : N} {delta : R} {Q : Set (R x EuclideanSpace R (Fin d))}
{L : ((R x EuclideanSpace R (Fin d)) -> R) ->
  (R x EuclideanSpace R (Fin d)) -> R}
(hdelta : 0 < delta /\ delta < 1)
(hL : ParabolicOperator L) (hQ : RegularParabolicDomain Q L)
(hcoeff : ParabolicOperatorCoefficientsHolder delta Q L) :
  forall f g, ParabolicHolderOn delta Q f -> ParabolicHolderOn (2 +
  delta) (closure Q) g ->
  exists u, ParabolicHolderOn (2 + delta) Q u /\

```

Natural language statement	Domain	GroundTruth
		ParabolicDirichletSolution Q L f g u /\nforall v, ParabolicHolderOn (2 + delta) Q v ->\nParabolicDirichletSolution Q L f g v ->\nSet.EqOn v u (Q union parabolicBoundary Q) := by\nsorry